Q20: How Much of an LLM's Internals Are Shared with Other Deep Learning Models?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) enumerate the components an LLM shares with the broader deep-learning family and the ones unique to it, (ii) explain why the shared core (differentiable modules, SGD, backpropagation, GPUs) keeps getting reused, (iii) identify which design choices in a Transformer are inherited from prior architectures and which are specific, and (iv) reason about transfer: when does knowledge from CNNs/RNNs/RL transfer to LLMs and when does it mislead.

1 The shared core: what every deep model has in common

Strip away the architecture debates and every deep learning model — convolutional network, recurrent network, Transformer, diffusion model, or policy network — is built from the same four ingredients [2]:

1. **A differentiable parametric function** f_θ composed of simple units (linear maps followed by nonlinearities), stacked in layers.
2. **A scalar loss** that scores how well f_θ performs the task.
3. **Gradient-based optimization** (SGD and its adaptive variants), which adjusts θ by following $-\nabla_\theta \mathcal{L}$ computed via *reverse-mode automatic differentiation* (backpropagation).
4. **Tensor hardware**: every model in the family is, at the bottom, a schedule of dense matrix multiplications and elementwise kernels — which is why one GPU ecosystem serves them all.

An LLM is a particular composition of these ingredients, not a departure from them. This is why "deep learning" knowledge — loss curves, learning-rate schedules, initialization, normalization, overfitting, regularization — transfers to LLMs mostly intact, and why the same tooling (PyTorch/JAX) runs everything in the family.

2 What the Transformer changed relative to its ancestors

The lineage is direct. The Transformer inherits *embeddings* from word2vec-era models, *attention* from seq2seq decoder–encoder attention [1], and the *residual stream + layer normalization* pattern from ResNet-era design [4]. Its specific contribution was removing recurrence: attention *alone* mixes information across positions, so the whole sequence is processed in parallel and gradients no longer travel through a time-chain [8]. Table 1 summarizes.

The practical reading: an LLM is a particular composition of inherited parts, not a new kind of machine — Transformers are a special case of "neural sequence model" that won the scaling race.

Component	Origin	Status in an LLM
SGD + backprop	1980s–90s	identical, scaled up
Embeddings	word2vec (2013)	identical idea: discrete token → vector
Attention	seq2seq (2014–15)	promoted from helper to the only mixer
Residual stream	ResNet (2015)	every sublayer writes additively into it
Layer norm	2016	standard (pre-norm placement in LLMs)
Parallel sequence mixing	Transformer (2017)	**new**: replaces recurrence
Next-token loss	language modeling	**new at scale**: one objective for everything

Table 1: Component lineage: an LLM is mostly inherited parts with two specific additions.

3 Where LLM internals differ from other family members

Four real differences matter for practice:

Scale and sparsity of parameters. A CNN's parameters live in small spatial filters shared across positions; an LLM's live in huge dense matrices (embeddings, attention projections, feed-forward blocks). LLMs today add *mixture-of-experts* sparsity [7], a trick with roots in the 1990s gating literature.

The loss is unsupervised and self-supplied. CNNs for vision were trained on labeled objectives; the LLM loss (predict the next token) is computed from the raw text itself. This changes everything about data pipelines and evaluation, while changing nothing about the optimizer.

Generation, not classification, is the interface. A classifier maps input → label once; an LLM maps prefix → distribution and then *iterates* its own output as new input. This autoregressive loop is where decoding strategies (sampling, temperature, beam search) enter — a concern inherited from RNN language models and machine translation.

RL enters after pre-training. Modern LLMs are stacked pipelines: pre-training (prediction) + supervised fine-tuning + preference optimization (DPO/RLHF) + RL with verifiable rewards [5, 6]. Other deep models sometimes add an RL fine-tuning stage (e.g., RL for robotics control), but the LLM stack's post-training is now a discipline of its own.

4 What this means for transfer of understanding

Because the core is shared, your CNN/RNN intuitions *mostly* carry over: vanishing gradients, normalization, initialization, learning-rate warmup, overfitting, data augmentation (for text: curation and de-duplication). Two systematic transfer mistakes to avoid:

- **Treating attention weights as explanations.** In CNNs, feature maps are at least spatially aligned with the input; attention weights are mixing coefficients, not evidence of "what the model looked at." Feature-attribution tools from vision do not transplant cleanly.

- **Assuming transfer learning works the same way.** In vision, fine-tuning a pretrained backbone on a small labeled set is the standard recipe. In LLMs, the analogous moves are prompting, in-context learning, LoRA-style parameter-efficient tuning, and full fine-tuning — with very different cost/benefit curves.

5 A map you can carry

One sentence: *an LLM shares the deep-learning skeleton (differentiable modules + SGD + backprop + tensor hardware) with every other deep model; it differs in its mixer (attention), its objective (next-token prediction at extreme scale), its interface (autoregressive generation), and its post-training stack.* When a new architecture appears (state-space models, diffusion language models), check the map: skeleton unchanged, mixer swapped, objective tweaked [3]. The skeleton is the stable part of the field.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[3] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *arXiv preprint arXiv:2312.00752*, 2023.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[5] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

[6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Chelsea Finn, and Sergey Levine. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[7] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.